

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ**
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ**
ВЫСШЕГО ОБРАЗОВАНИЯ
**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. ШУХОВА»**
(БГТУ им. В.Г. Шухова)



ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И УПРАВЛЯЮЩИХ СИСТЕМ

КУРСОВОЙ ПРОЕКТ

по дисциплине: Конструирование программного обеспечения
тема: «Разработка сервиса хранения, версионирования и распространения
аудиоданных»

Автор работы _____ Пахомов Владислав Андреевич ПВ-223
(подпись)

Руководитель проекта _____ Заикин Александр Олегович
(подпись)

Оценка _____

Белгород 2025 г.

Оглавление

1	Введение	3
2	Анализ предметной области. Создание базовой структуры проекта	4
2.1	Анализ предметной области	4
2.2	Структура проекта	6
3	Декомпозиция и визуализация программного проекта	6
4	Методы управления	8
4.1	Выбор и обоснование методологии	8
5	Организация рабочего пространства управления проектом . . .	10
5.1	Принципы организации рабочего пространства	10
5.2	Инструменты организации рабочего пространства	10
6	Система контроля и мониторинга реализации проекта	11
6.1	Философия и принципы системы контроля	11
7	Оценка затрат на реализацию и внедрение	11
7.1	Оценка затрат на реализацию	11
7.2	Оценка затрат на внедрение	12
8	Вывод о проделанной работе	12
9	Список источников и литературы	14

1 Введение

Одной из задач при проектировании любого из решений на сегодняшний день является организация работы. Это, в том числе, может относиться и к аудиоданным. В данной работе будет рассмотрен план развития сервиса для разрешения такой задачи.

Цель курсовой работы заключается в приобретении практических навыков создания и управления программными проектами. Для достижения этой цели необходимо решить следующие задачи:

- Провести анализ предметной области
- Декомпонировать и визуализировать задачу
- Выбрать наиболее подходящий метод управления для проектирования сервиса
- Организовать рабочее пространство управления проекта
- Выбрать подходящую систему контроля и мониторинга реализации проекта
- Оценить затраты на реализацию и внедрение
- Провести анализ внешних и внутренних рисков. Предусмотреть способы их решения

2 Анализ предметной области. Создание базовой структуры проекта

2.1 Анализ предметной области

Процесс разработки музыкальных продуктов часто сопряжён с постоянными изменениями в составляющих аудиоданных. Было бы полезно иметь возможность быстро проследить, как изменялась каждая из версий аудиоданных. Часто возвращение к старым идеям, кроме того, может дать толчок для развития в текущей итерации. Также было бы полезно иметь возможность распространения определённой версии аудиотрека для совместной работы. Возможно, стоит рассмотреть возможность публикации полученных результатов в стриминговых сервисах.

Функциональные требования:

- Безопасность и управление доступом
 - Аутентификация через Keycloak
 - Управление порталными ролями (User, Manager, Composer, Admin)
- Управление сущностями автора
 - Возможность создания, редактирования, удаления и просмотра сущности автора
- Управление сущностями группы
 - Возможность создания, редактирования, удаления и просмотра сущности группы
 - Обеспечение целостности группы
- Управление сущностями песнями

- Возможность создания, редактирования, удаления версий песен (версионированные данные: название, описание, BPM, тональность, слова, файлы, тэг, публичность)
- Переключение между версиями песен
- Возможность комментирования песен
 - * Возможность создания, редактирования, удаления и просмотра комментариев, прикрепленных к песне
- Хранение файлов
 - Реализация способа архивации/деархивации аудиофайлов с возможностью потоковой распаковки
- Управление сущностями плейлистов
 - Возможность создания, редактирования, удаления, просмотра плейлистов
 - Возможность поделиться плейлистом с учётом публичности плейлиста
 - Возможность создания продвинутых фильтров для подбора конкретной версии песни
- Воспроизведение
 - Возможность воспроизведения треков из плейлиста
 - Воспроизведение нескольких треков
 - Корректировка аудиопотока с простейшими эффектами, вроде эквалайзера
- Интеграция со стриминговыми сервисами

2.2 Структура проекта

Архитектура проекта была выбрана микросервисной. API Gateway будет содержать бизнес-логику для обработки ролевой модели, сущностей, получения данных от клиента и их валидацией. Для стриминга и архивации/деархивации аудиоданных будет использован отдельный сервис.

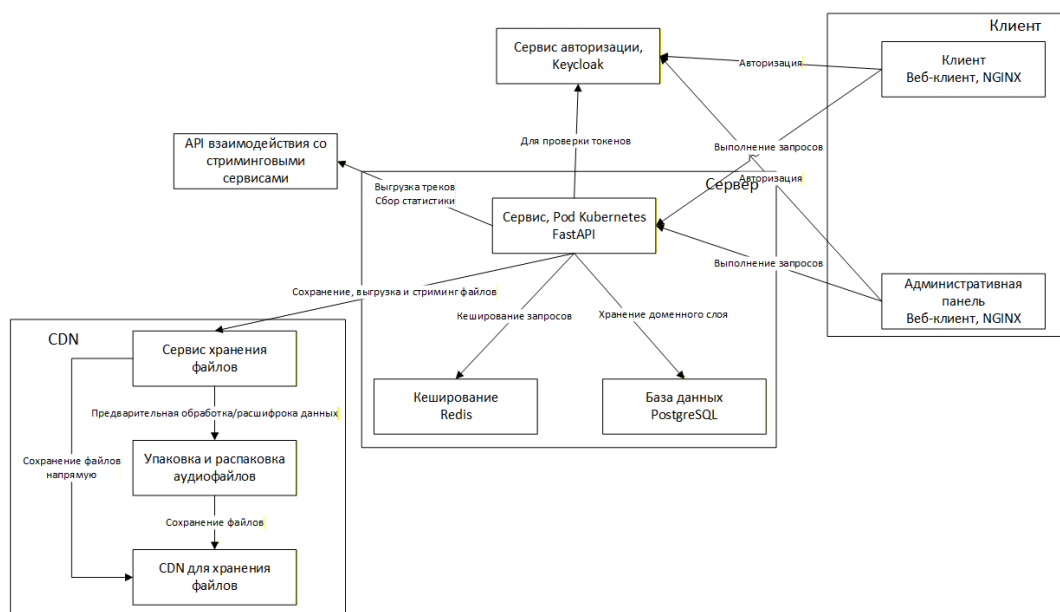
В качестве стека для первого компонента был выбран стэк из FastAPI, Pydantic, SQLAlchemy, что позволяет обеспечить быструю гибкую разработку.

Второй сервис требует большой скорости и стабильности, поэтому в качестве стека был выбран стек язык программирования Rust.

Инфраструктура может включать в себя сервис авторизации Keycloak. Для развёртывания и масштабирования необходимо использовать Kubernetes и образы Docker. В качестве базы данных выбран PostgreSQL. Для снижения нагрузки на взаимодействие по сети с PostgreSQL можно подключить Redis.

3 Декомпозиция и визуализация программного проекта

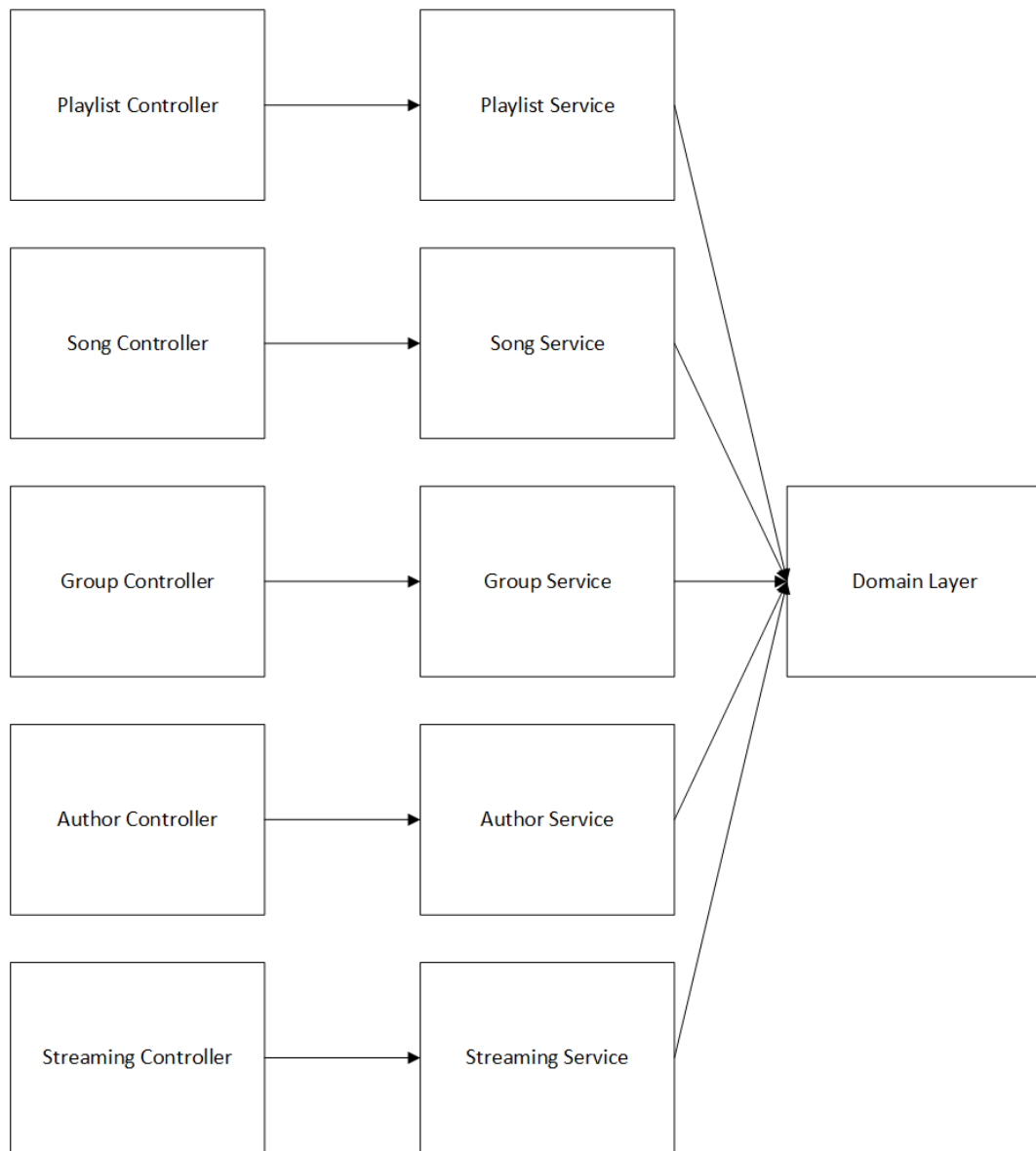
Сервис состоит из нескольких микросервисов, выполняющих свою задачу. Архитектуру микросервисов можно представить в следующем виде:



Структура клиента:



Структура сервера:



4 Методы управления

4.1 Выбор и обоснование методологии

В качестве метода управления был выбран Agile. Данный выбор был обоснован следующими факторами:

- Большая гибкость. Проект является по большей части экспериментальным, множество из предложенных ”фичей” проекта могут оказаться ненужными и бесполезными. На первых этапах для проработки концепции можно будет добавить небольшое количество разработчиков и, если проект окажется успешным после опробования,

можно будет подключить дополнительное количество разработчиков.

- Бюджетность. Для проекта не выделяются большие средства на первых этапах. Agile позволит подготовить MVP для проверки идеи, и если проект окажется успешным, его можно будет развивать дальше.
- Отзывчивость. Обратная связь крайне важна для проекта, требования могут меняться очень часто. Гибкая методология позволяет адаптироваться к новым требованиям.

Методология Agile предполагает разбиение задачи на несколько более мелких подзадач. Такие подзадачи позволяют выделить приоритетность и организовать работу. На первых этапах необходимо начать разработку с минимально живого сервиса, который будет позволять пользователям выстраивать полезные бизнес данные. Задачи подключения к стриминговым сервисам, оптимизации хранения наименее важны, и их можно отнести к более поздним эпикам. Итого получаем:

Эпик 1:

- Авторизация с помощью Keycloak
- Версионная модель песен

Эпик 2:

- Группы
- Авторы
- Плейлисты

Эпик 3:

- Продвинутый проигрыватель

Эпик 4:

- Оптимизация сервиса хранения данных
- Интеграция с стриминговыми сервисами
- Подключение сервиса кеширование, рост сервиса в ширину
- Дальнейшая оптимизация

5 Организация рабочего пространства управления проектом

5.1 Принципы организации рабочего пространства

Критически важная информация должна быть доступна участникам, храниться в одном месте, процессы, особенно критически важные, должны быть автоматизированы, а также должна быть защищена от потенциальных угроз безопасности.

5.2 Инструменты организации рабочего пространства

В качестве системы контроля версий используется Github с подключением собственных раннеров. В проекте у членов команды должна быть своя роль, у ролей должны быть настроены разрешения и запреты. Каждый запрос на слияние должен проходить ревью, а слияние должно производиться ответственным участником команды (тимлидом).

Все ключевые данные должны храниться в отдельном волте. Получение ключей при сборке в Github Actions должно выполняться через него.

Код должен иметь единый стиль и форматирование. Настройку форматирования кода передать DevOps члену команды.

Для хранения полученных образов можно использовать Github Registry и в дальнейшем self-hosted решение.

Развёртка приложений и доставка должна происходить автоматически, контейнерезированно. Docker контейнеры должны быть развёрнуты в Kubernetes.

6 Система контроля и мониторинга реализации проекта

6.1 Философия и принципы системы контроля

Система контроля и мониторинга реализации проекта строится на концепции управления и получения обратной связи по пунктам, которые измерить важно, а не легко. Контроль осуществляется на операционном уровне - то как хорошо работает код и насколько он качественен, - тактическом - то, как хорошо мы укладываемся в сроки, - и стратегическом - то, насколько хорошо полученный продукт отвечает бизнес требованиям. Все метрики соответствуют концепции SMART. Они конкретные, измеримые, достижимые, важные и ограниченные по времени.

7 Оценка затрат на реализацию и внедрение

Предположим, что проект получил положительный отклик, пользователи хотят все предложенные нами функциональные требования, и теперь можно переходить к реализации всех задач. Нам необходимо развернуть проект за 1 год

7.1 Оценка затрат на реализацию

- Backend-разработчик (Python): Весь цикл проекта. 130000 руб/месяц
- Frontend-разработчик (Веб): Весь цикл проекта. 115000 руб/месяц
- Дизайнер: 2 месяца. 100000 руб/месяц.

- DevOps: Весь цикл проекта. 130000 руб/месяц.
- Низкоуровневый ML-разработчик (Rust): 6 месяцев. 175000 руб/месяц.
- Менеджер: Весь цикл проекта. 100000 руб/месяц.

Итого: 6950000 руб. - чистые расходы на зарплату.

7.2 Оценка затрат на внедрение

Аренда

включает в себя раннеры (около 3000 руб/месяц) а также серверы, которые будут оркестрировать сервисы, поддерживать микросервисную архитектуру и запускать сервисы (около 15000 руб/месяц).

Итого: 18000 руб/месяц.

8 Вывод о проделанной работе

Протокол FTR, пусть и не является на сегодняшний день рекомендованным и безопасным вариантом передачи данных, всё ещё может использоваться в некоторых специфических сценариях, вроде работы с устаревшими системами а также для более быстрой передачи данных в локальных малонагруженных сетях.

Компьютерные сети развивались стремительно, особенно в конце XX века, и это можно заметить по ходу развития протокола передачи данных. Он дополнялся и расширялся, многие вводили предложения и экспериментировали, чтобы на сегодняшний день мы имели эффективный безопасный способ обмена данными, позволяющий нам выполнять множество задач. На сегодняшний день технологии передачи файлов замедлили своё развитие в сфере передачи файлов из файловой системы в файловую систему компьютеров. Для решения этой задачи уже был найден подходящий безопасный и рабочий протокол SFTP. Однако с

решением одних задач приходят новые проблемы, и задача современных разработчиков - разрешение этих проблем. Это возможно с использованием лучшего от старых подходов и применения новых идей для решения новых задач.

Разработанная библиотека CtrlFTP позволяет создавать гибкий расширяемый FTP сервер, который может быть полезен для решения специфических задач бизнеса, вроде расширенной поддержки форматов и кодировок, файловой интерпретации нефайловых данных (записи в базе данных).

Получившийся клиент позволяет работать с RFC-959 совместимыми серверами, удалять и копировать файлы между компьютерами, создавать папки, переименовывать. Он также допускает работу в пассивном и активном режимах.

Ссылка на репозиторий с кодом:
<https://github.com/IAmProgrammist/CtrlFTP>

9 Список источников и литературы

1. A File Transfer Protocol [Электронный ресурс]
Режим доступа: <https://datatracker.ietf.org/doc/html/rfc114>
2. The Use of 'Set Data Type' Transaction in File Transfer Protocol [Электронный ресурс]
Режим доступа: <https://datatracker.ietf.org/doc/html/rfc294>
3. The File Transfer Protocol [Электронный ресурс]
Режим доступа: <https://datatracker.ietf.org/doc/html/rfc354>
4. File Transfer Protocol [Электронный ресурс]
Режим доступа: <https://datatracker.ietf.org/doc/html/rfc454>
5. File Transfer Protocol (FTP) [Электронный ресурс]
Режим доступа: <https://datatracker.ietf.org/doc/html/rfc959>
6. Internationalization of the File Transfer Protocol [Электронный ресурс]
Режим доступа: <https://datatracker.ietf.org/doc/html/rfc2640>
7. FTP Security Extensions [Электронный ресурс]
Режим доступа: <https://datatracker.ietf.org/doc/html/rfc2228>
8. The Secure Shell (SSH) Transport Layer Protocol [Электронный ресурс]
Режим доступа: <https://datatracker.ietf.org/doc/html/rfc4253>
9. Hypertext Transfer Protocol – HTTP/1.1 [Электронный ресурс]
Режим доступа: <https://datatracker.ietf.org/doc/html/rfc2068>
10. Hypertext Transfer Protocol – HTTP/1.1 [Электронный ресурс]
Режим доступа: <https://datatracker.ietf.org/doc/html/rfc2616>
11. Documentation | Adoptium [Электронный ресурс]
Режим доступа: <https://adoptium.net/docs/>

12. Getting started with Hibernate ORM [Электронный ресурс]

Режим

доступа:

<https://hibernate.org/orm/documentation/getting-started/5.x/>

13. Qt for Python

Режим доступа: <https://doc.qt.io/qtforpython-6/index.html>

14. ftplib — FTP protocol client

Режим доступа: <https://docs.python.org/3/library/ftplib.html>

15. SFTP vs. FTP: Understanding the Difference

Режим доступа: <https://www.sharetru.com/blog/sftp-vs-ftp-understanding-the-difference>

16. Telnet Protocol Specification

Режим доступа: <https://datatracker.ietf.org/doc/html/rfc854>